COP3330C Module 10 Programming Assignment 2: Applying the Iterator Pattern with the Java Optional Type

In this assignment, you will practice using the **Optional** type (which is actually a generic class) in an application which applies the **Iterator** pattern. Optional provides a way to handle possible absence of a value in a safe manner. The Iterable interace specifies how to iterate through a collection of objects, usually using an enhanced for loop.

This assignment also introduces the use of **Git** for managing applications and their relationship to GitHub repositories. Git is useful for local version control and useful for submitting folders to GitHub. It also makes use of the IntelliJ Ultimate IDE; this tool was specified as a technology requirement in the course syllabus. If you have not obtained your academic account from JetBrains yet you will need to do that before completing this assignment. Links for these tools are provided in the course syllabus and below for your convenience.

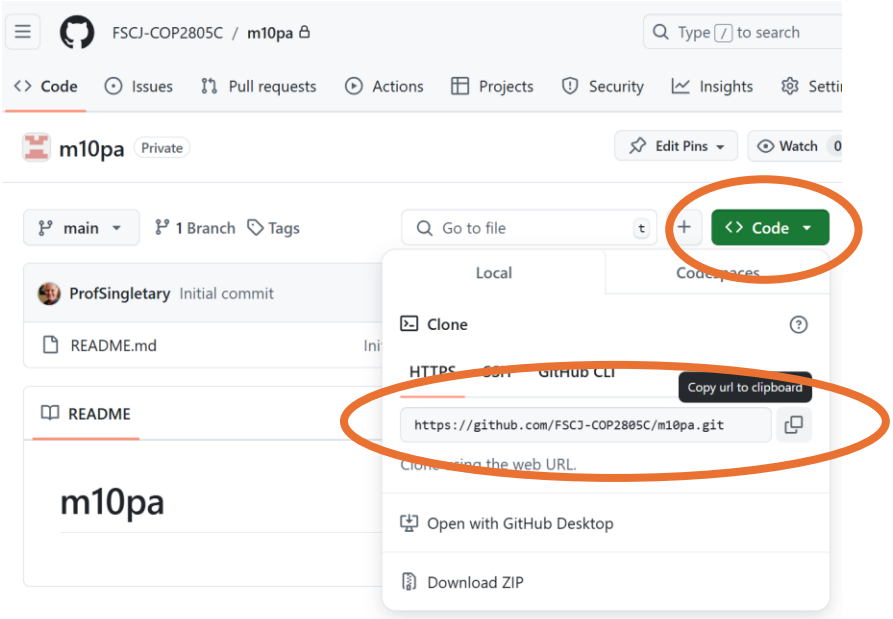
If you are unable to install the required tools on a personal computer you will need use the Horizon Academic_IT_W11 system, where IntelliJ IDE and Git are preinstalled. Please note that your instructor cannot help with installation or configuration issues on a personal system.

- **IntelliJ IDEA Ultimate https://www.jetbrains.com/idea/**
    - (academic license required: https://www.jetbrains.com/lp/leaflets-gdc/students/)
- **Git https://git-scm.com/**
    - Downloads:
        - https://git-scm.com/download/win
        - https://git-scm.com/download/mac
        - **https://git-scm.com/download/linux**

Instructions:

Create a program that manages a directory of students using a Student class. Implement an iterator for the student list and use the Optional class to handle cases where a student's information might not be found.

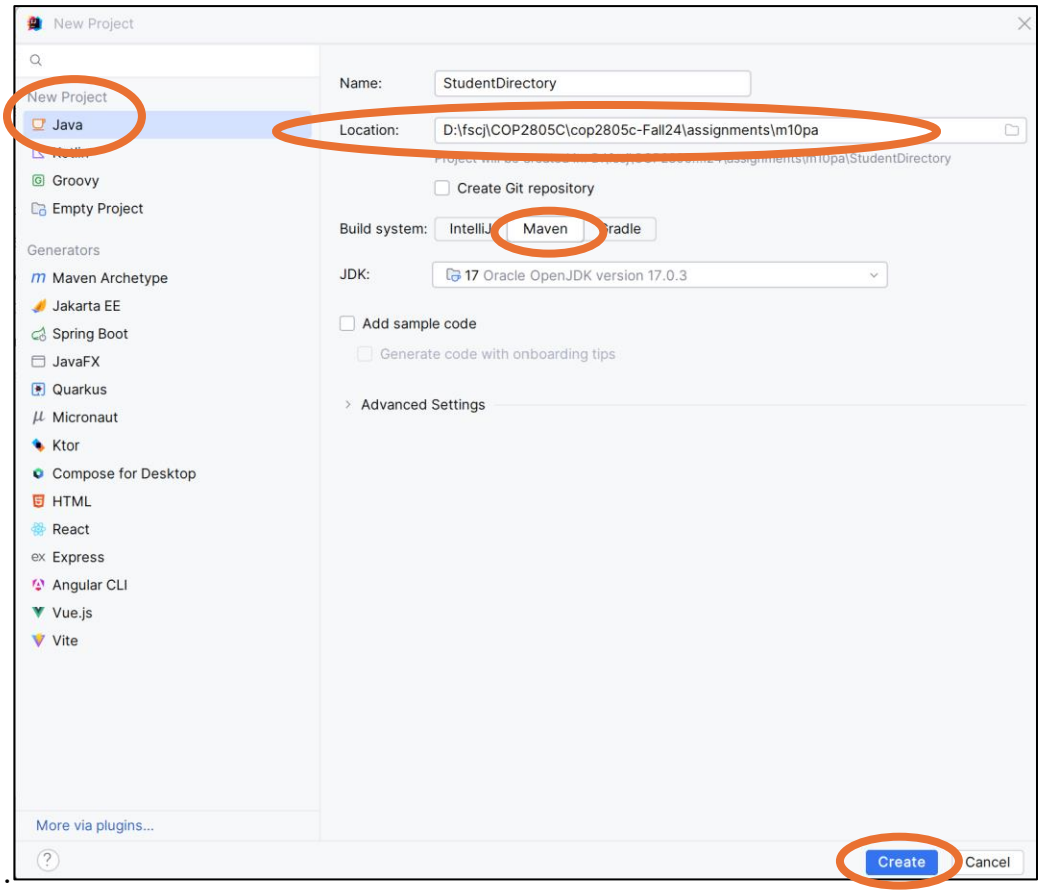
Start by accepting the GitHub Classroom assignment invitation in Canvas. After your personal repository has been created, clone the repo on a local system by copying the link from the Code dropdown in the repo, then cloning it locally (NOTE: you will see references to COP2805C in the screenshots in this walkthrough, your repository will use COP3330C):



Paste the link on the command line after entering **git clone**:

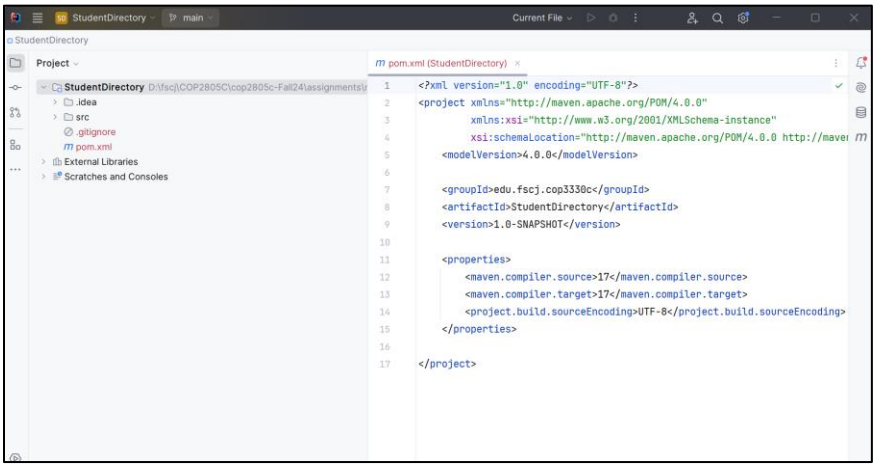
```
D:\fscj\COP2805C\cop2805c-Fall24\assignments>git clone https://github.com/FSCJ-COP2805C/m10pa.git
Cloning into 'm10pa'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\fscj\COP2805C\cop2805c-Fall24\assignments>
```

Create a new Java project in IntelliJ. Enter your cloned repo location as the project folder and **Maven** as the build system. Be sure the JDK is specified, then press Create:

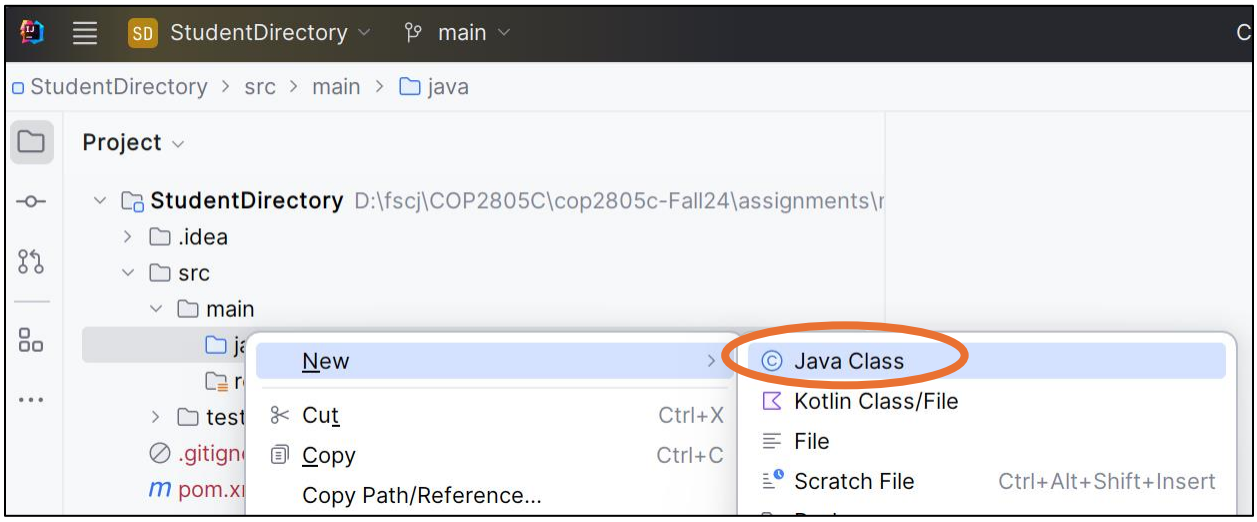


.

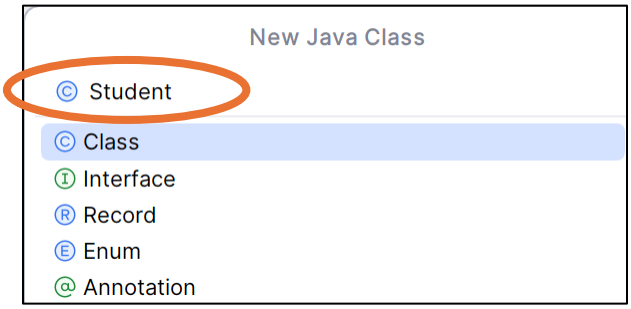
IntelliJ starts in the project folder and will open the "pom.xml" file used to configured the Maven build parameters. You can close this file, it won't be needed for this project.



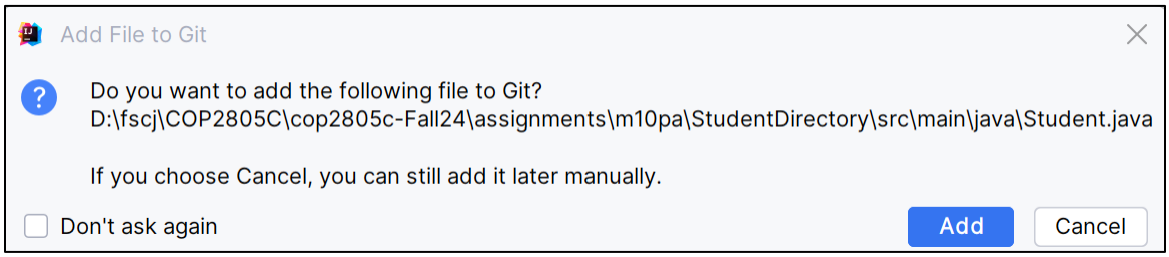
Expand **src/main/java** in the project navigation pane and select **New/Java Class**



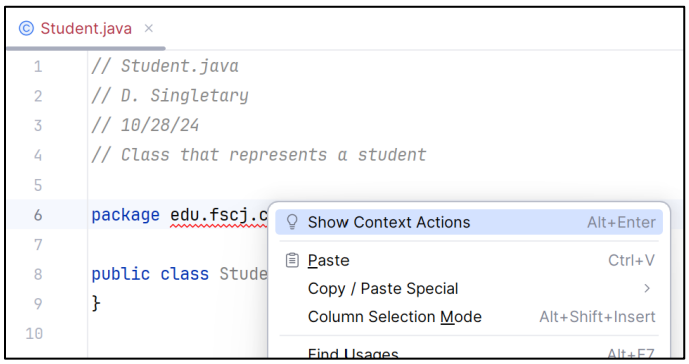
Enter **Student** as the class name:

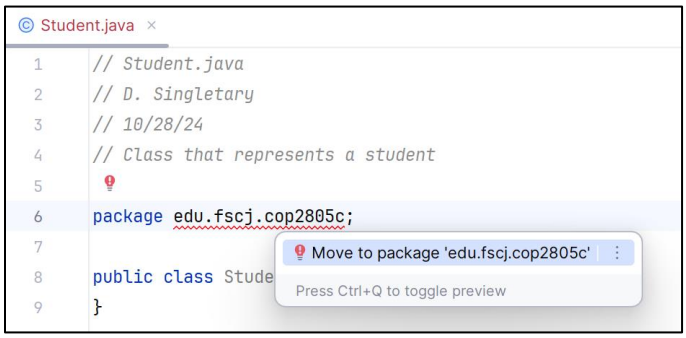


If you want IntelliJ to manage your local Git repository you can click Add here, otherwise just Cancel. I usually just Cancel and manage the files manually later, which is what I will do for this walkthrough.

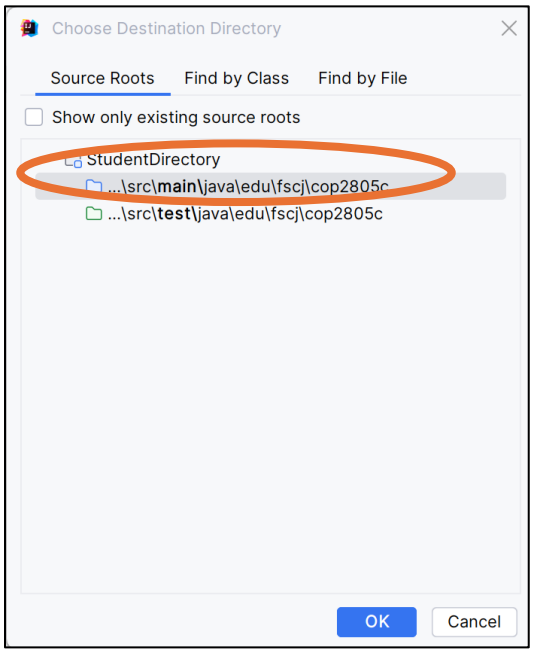


Start creating the Student class. Include your ID header and the package specification **package edu.fscj.cop3330c;** Notice that IntelliJ displays a red "wavy line to indicate the package doesn't exist yet, right-click on the line and select **Show Context Actions**, then select **Move to package ...**

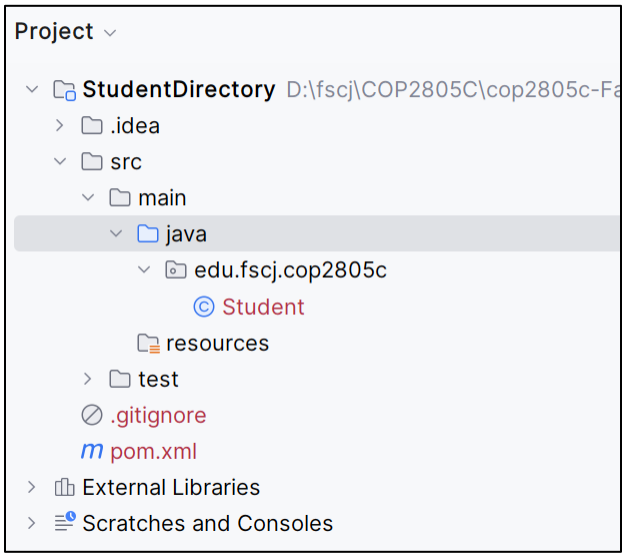




Choose the main folder (not the test folder) as the location for the package:



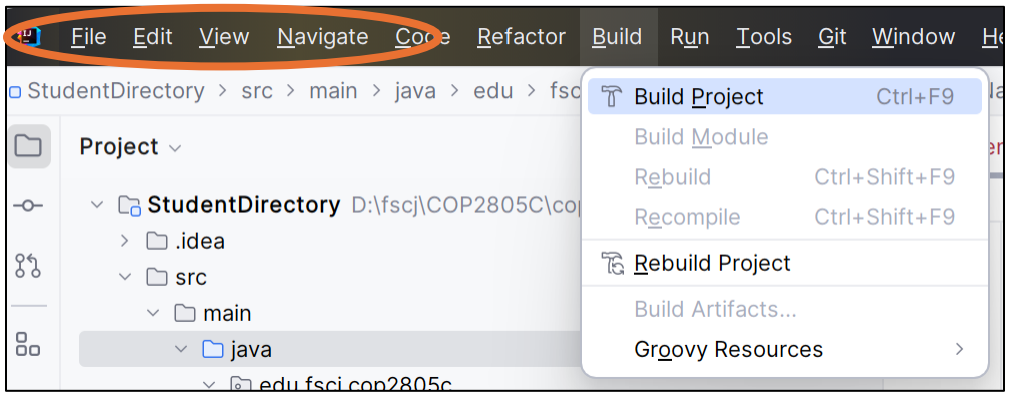
IntelliJ then creates the package and moves the file into the correct location:



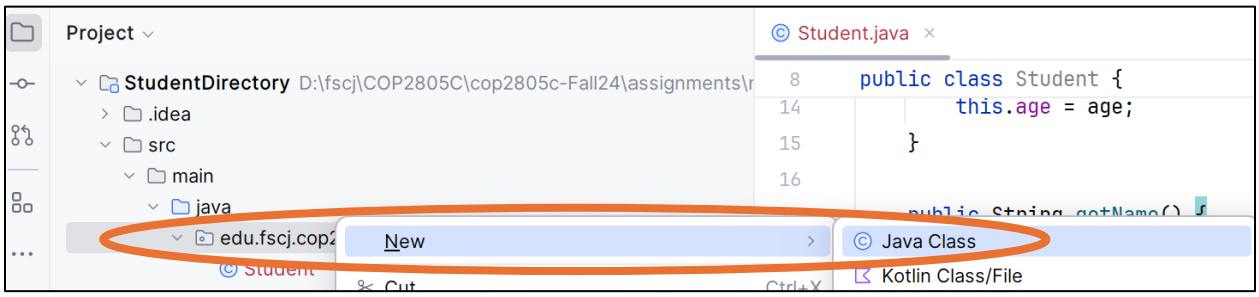
Continue entering the code for the Student class:

```
public class Student {
    private String name;
    private Integer age;

    public Student(String name, Integer age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public Integer getAge() {
        return age;
    }

    @Override
    public String toString() {
        return "Student{name='" + name + "', age=" + age + "}";
    }
}
```

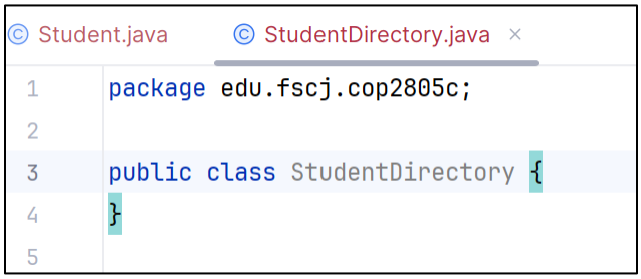
At this point you can run a Build to verify your code is correct:



Next, right click on the package and add a new class named StudentDirectory.



IntelliJ will now add the package specification for you:



Add your ID header and continue by adding the StudentDirectory code

```
// StudentDirectory.java
// D. Singletary
// 10/28/24
// Class/application which represents a student directory

package edu.fscj.cop3330c;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Optional;

class StudentDirectory implements Iterable<Student> {
    private List<Student> students;
```

```
public StudentDirectory() {
    this.students = new ArrayList<>();
}

public void addStudent(Student student) {
    students.add(student);
}

public Optional<Student> findStudentByName(String name) {
    for (Student student : students) {
        if (student.getName().equalsIgnoreCase(name)) {
            return Optional.of(student);
        }
    }
    return Optional.empty();
}

@Override
public Iterator<Student> iterator() {
    return students.iterator();
}

public static void main(String[] args) {
    StudentDirectory directory = new StudentDirectory();

    // Adding students to the directory
    directory.addStudent(new Student("Alice", 20));
    directory.addStudent(new Student("Bob", 22));
    directory.addStudent(new Student("Charlie", 19));

    // Iterating over the students using Iterator pattern
    System.out.println("Student List:");
    for (Student student : directory) {
        System.out.println(student);
    }

    // Finding a student by name using Optional
    String searchName = "Bob";
    Optional<Student> studentOpt = directory.findStudentByName(searchName);

    // Handling the Optional result
    studentOpt.ifPresentOrElse(
            student -> System.out.println("Found student: " + student),
            () -> System.out.println("Student with name " +
                    searchName + " not found.")
    );

    // Searching for a non-existing student
```

```
        String nonExistingName = "David";
        Optional<Student> nonExistingStudentOpt =
                    directory.findStudentByName(nonExistingName);
        nonExistingStudentOpt.ifPresentOrElse(
                student -> System.out.println("Found student: " + student),
                () -> System.out.println("Student with name " +
                        nonExistingName + " not found.")
        );
    }
}
```

Notice the use of Optional in this class, which handles the possibility that a student with the specified name might not be found. This is a safer and cleaner way to deal with potentially missing values instead of checking for null.

The iterator provides a way for the program to iterate over the StudentDirectory class without exposing the internal List implementation.

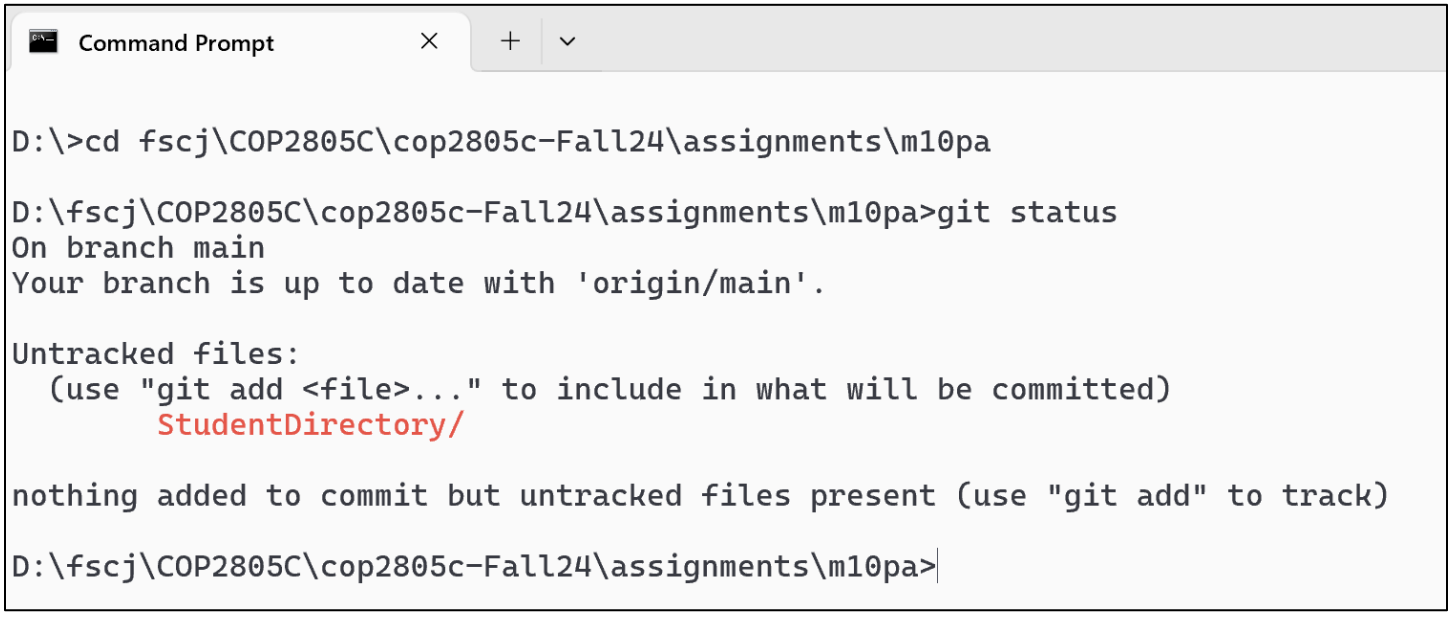
Build and run the program to verify correct operation, then close the project from the file menu.

Expected output:

Student List:
Student{name='Alice', age=20}
Student{name='Bob', age=22}
Student{name='Charlie', age=19}
Found student: Student{name='Bob', age=22}
Student with name David not found.

Now we will use Git to commit the project solution.

Open a Command Prompt from the taskbar and navigate to your project folder with **cd**, then run **git status**:



The folder in red indicates there are files that have not been “staged” to the repo yet. Run **git add .** (there’s a space and dot at the end of that command) then run git status again:

```
D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>git add .
warning: in the working copy of 'StudentDirectory/.gitignore', LF will be replaced by CRLF the next time Git touches
warning: in the working copy of 'StudentDirectory/pom.xml', LF will be replaced by CRLF the next time Git touches it

D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   StudentDirectory/.gitignore
        new file:   StudentDirectory/.idea/.gitignore
        new file:   StudentDirectory/.idea/encodings.xml
        new file:   StudentDirectory/.idea/misc.xml
        new file:   StudentDirectory/.idea/uiDesigner.xml
        new file:   StudentDirectory/.idea/vcs.xml
        new file:   StudentDirectory/pom.xml
        new file:   StudentDirectory/src/main/java/edu/fscj/cop2805c/Student.java
        new file:   StudentDirectory/src/main/java/edu/fscj/cop2805c/StudentDirectory.java


D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>
```

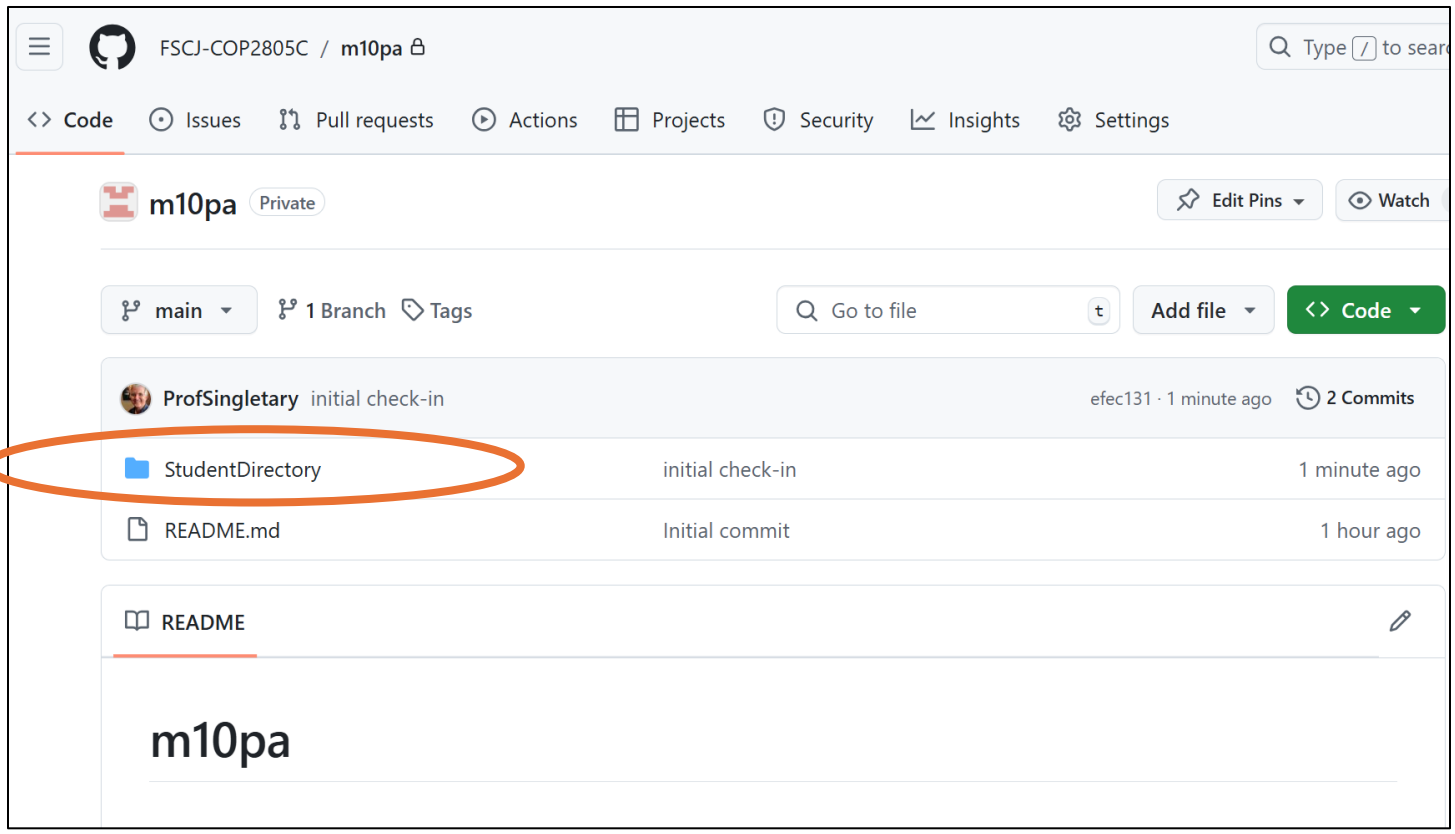
The green files indicate we are now ready to commit to the repo.

Run **git commit -m “your commit comment”** . Replace “your commit comment” with a meaningful comment.

Then run **git push:**

```
D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>git commit -m "initial check-in"
[main efec131] initial check-in
 9 files changed, 316 insertions(+)
 create mode 100644 StudentDirectory/.gitignore
 create mode 100644 StudentDirectory/.idea/.gitignore
 create mode 100644 StudentDirectory/.idea/encodings.xml
 create mode 100644 StudentDirectory/.idea/misc.xml
 create mode 100644 StudentDirectory/.idea/uiDesigner.xml
 create mode 100644 StudentDirectory/.idea/vcs.xml
 create mode 100644 StudentDirectory/pom.xml
 create mode 100644 StudentDirectory/src/main/java/edu/fscj/cop2805c/Student.java
 create mode 100644 StudentDirectory/src/main/java/edu/fscj/cop2805c/StudentDirectory.java

D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>git push
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 16 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (19/19), 4.00 KiB | 2.00 MiB/s, done.
Total 19 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/FSCJ-COP2805C/m10pa.git
   d849f34..efec131  main -> main

D:\fscj\COP2805C\cop2805c-Fall24\assignments\m10pa>
```

Your project files have now been committed to the repository, verify this by navigating to it using your browser.


FSCJ-COP2805C / m10pa
Type / to sear
Code
Issues
Pull requests
Actions
Projects
Security
Insights
Settings
m10pa
Private
Edit Pins
Watch
main
1 Branch
Tags
Go to file
t
Add file
Code
ProfSingletary initial check-in
efec131 · 1 minute ago
2 Commits
StudentDirectory
initial check-in
1 minute ago
README.md
Initial commit
1 hour ago
README
m10pa